CampusCycle

A Centralized Web Platform for Campus Resource Sharing & Exchange

Tech Stack: React.js • Spring Boot • MySQL • JWT Security

Name	G. Sohan Reddy
Roll No.	24EG105P24
Branch	CSE

1. Problem Statement

University students regularly acquire academic and personal resources such as textbooks, laboratory equipment, calculators, drawing kits, project components, and hostel essentials — many of which are needed only for a single course, semester, or short period of time. The traditional approach to exchanging these resources relies heavily on fragmented WhatsApp groups, personal contacts, hostel notice boards, or informal word of mouth.

This unstructured process leads to several recurring problems:

- **Inefficiency** — It takes too long to find someone who has the exact item you need, or someone who needs the item you no longer use.
- **Lack of Visibility** — Listings posted in scattered chat groups are easily buried and missed by the students who would actually want them.
- **No Trust Mechanism** — There is no reliable way to verify that the other party is a genuine, verified student before proceeding with an exchange.
- **No Centralization** — There is no single, dependable platform serving the entire campus for buying, selling, renting, borrowing, or donating resources.
- **Buy-Only Bias** — Existing channels and general marketplaces are built around one-off buying and selling, with no structured support for short-term needs like renting or borrowing.

2. Proposed Solution

We propose **CampusCycle**, a centralized web-based resource exchange platform built specifically for the campus community. The system allows verified students to:

- **List Resources** — Post an item for Sell, Rent, Borrow, or Donate, along with photos, category, condition, and price/duration where applicable.
- **Request Items** — Post what they currently need, even if no matching listing exists yet.
- **Search and Filter** — Easily search for items by category (Books, Electronics, Lab Equipment, etc.), listing type, price, and availability.

- **Secure Exchange** — Follow a structured process where both parties confirm the transaction and meet at a predefined campus pickup location.
- **Dashboard** — View a clear, consolidated feed of available listings and open requests across the campus.

3. Technology Stack — Where & How It Is Used

This section gives a simple, project-specific view of each technology — what it does in this system, not a general definition. The diagram at the end shows how one request flows through the full stack.

3.1 React.js

- Builds the pages users see — dashboard, listing form, request form, and profile.
- Fetches data from the backend and displays it as the live feed of available listings and open requests.
- Stores the login token after sign-in and attaches it to every request.
- Moves between pages (login → dashboard → listing/request/profile) without reloading the browser.

3.2 Spring Boot

- Receives requests from React — login, create a listing, submit a request, initiate a transaction — through REST APIs.
- Applies the business rules, e.g. a new listing starts as **AVAILABLE**, and a transaction needs an available listing.
- Talks to the MySQL database to save and retrieve records.
- Sends the result back to React as a JSON response.

3.3 MySQL

- Stores all data permanently across four tables: users, listings, requests, and transactions.
- Answers the queries behind each screen, e.g. fetching only **AVAILABLE** listings for the dashboard.
- Foreign keys keep listings, requests, and transactions correctly linked to the right user.

3.4 JWT & Spring Security

- Issues a signed token when a user logs in successfully.
- Checks this token on every request to confirm who the user is.
- Passwords are hashed before being stored, so raw passwords are never saved.

3.5 End-to-End Flow

Figure — how one request moves through the stack, from the UI to the database and back.

4. Frontend Flow (React)

The frontend is built using React.js to deliver a fast, single-page application (SPA) experience.

Core Screens

- **Landing Page / Dashboard** — Displays a feed of recent listings; users toggle between Sell, Rent, Borrow, and Donate tabs.
- **Authentication Flow** — Registration and login for students, verified using college email IDs.
- **Listing Flow** — User clicks "List an Item" → selects type (Sell/Rent/Borrow/Donate) → fills out a form (title, category, condition, price/duration, image upload) → submits.
- **Request Flow** — User clicks "Request an Item" → describes what they need and by when → the request stays open until fulfilled or expired.
- **Transaction Flow** — User finds a matching listing → clicks "Reserve" → contacts the owner → confirms the exchange at a campus pickup point.
- **Profile Page** — Users manage their listings and requests, and track the status of their transactions.

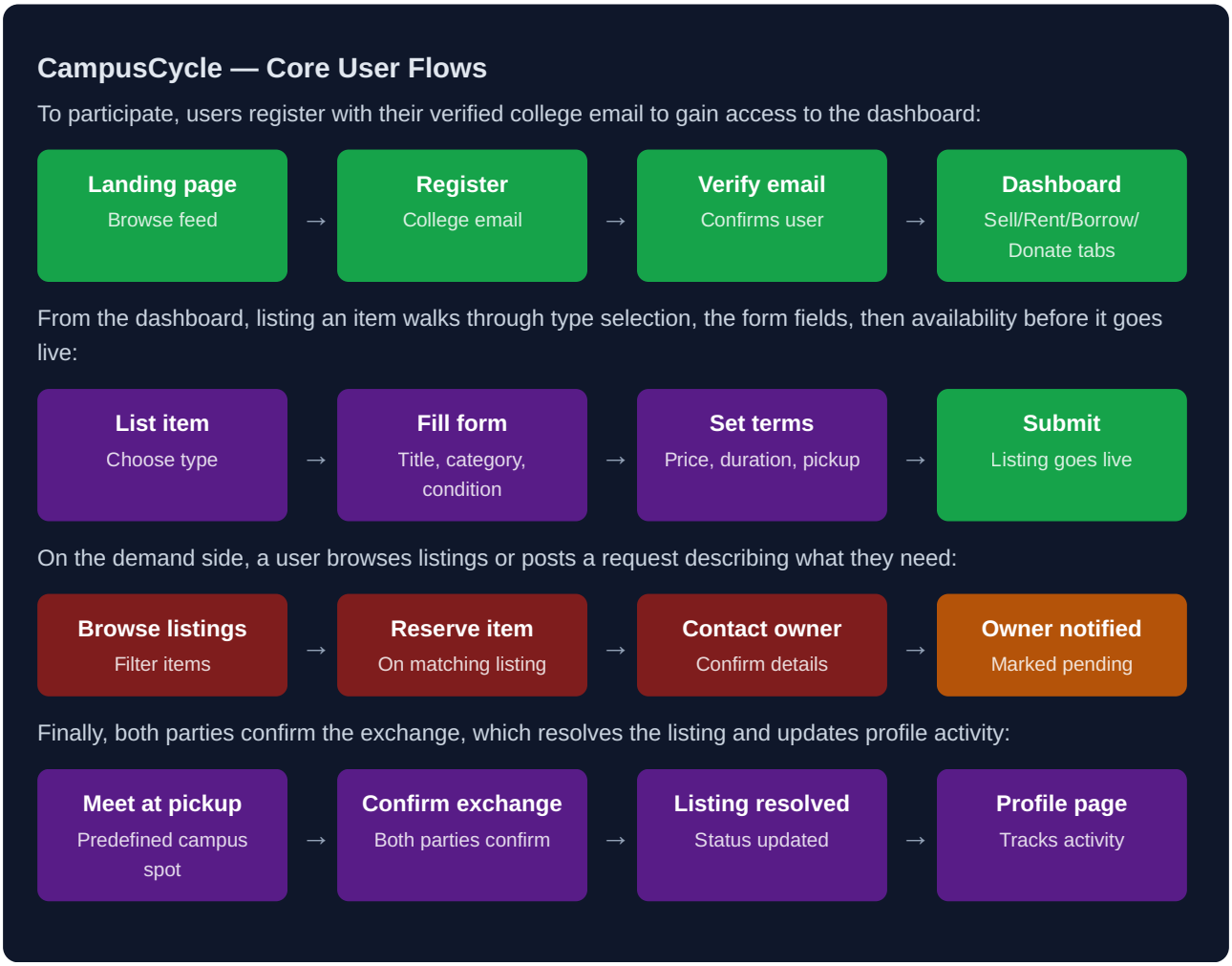

Figure 1 — Core user flows: registration, listing, requesting, and exchange resolution.

5. Backend Flow (Spring Boot)

The backend is a RESTful API built with Spring Boot.

Architecture

- **Controllers** — Handle incoming HTTP requests from the React frontend.

- **Services** — Contain the core business logic, such as listing status updates and request-matching checks.
- **Repositories** — Interface with the MySQL database using Spring Data JPA.
- **Security** — Spring Security with JWT (JSON Web Tokens) ensures only authenticated users can post listings or requests.
- **Image Handling** — Listing images are stored locally or in a cloud service, with the URL saved in the database.

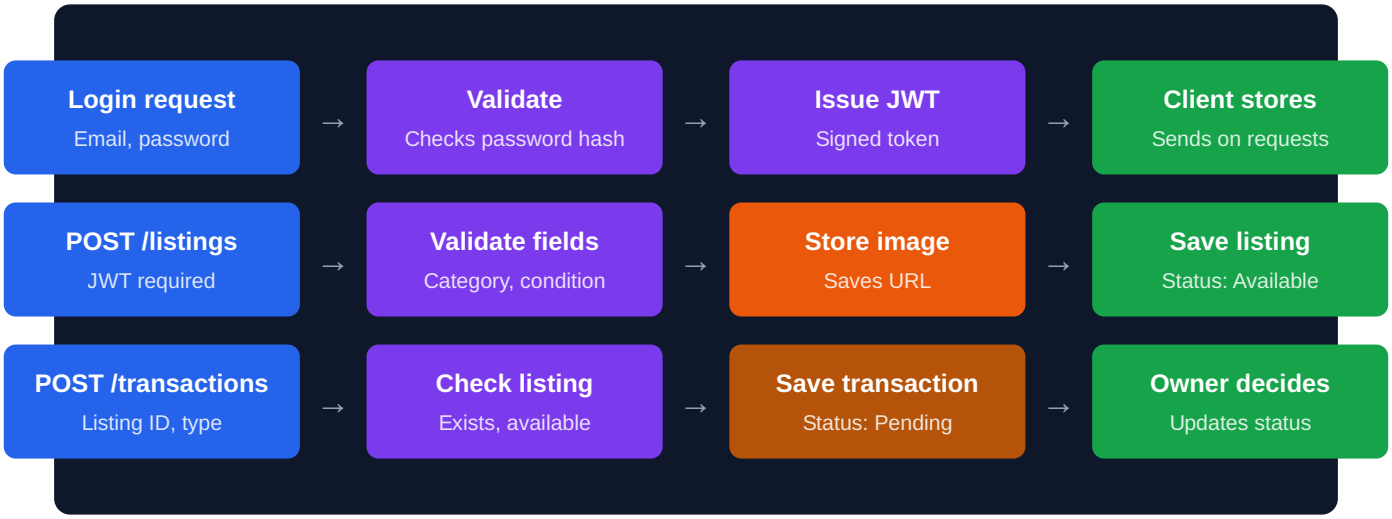

Figure 2 — Backend architecture and request flows for login, listing creation, and transactions.

6. Database Schema (MySQL)

The system uses a structured relational database consisting of four tables. A user can post many listings and many requests, and can be involved in many transactions; a listing can receive multiple transaction attempts, though realistically only one is completed.

Users Table — one row per student or admin

Field	Type / Notes
user_id	Primary Key, Auto Increment
full_name	Varchar
email	Varchar, Unique — preferably college email
password_hash	Varchar
phone_number	Varchar
role	Enum: STUDENT, ADMIN
created_at	Timestamp

Listings Table — one row per item posted for sale, rent, borrowing, or donation

Field	Type / Notes

listing_id	Primary Key, Auto Increment
user_id	Foreign Key → users.user_id (the owner)
title	Varchar
description	Text
category	Enum: BOOKS, ELECTRONICS, LAB_EQUIPMENT, FURNITURE, SPORTS, OTHER
listing_type	Enum: SELL, RENT, BORROW, DONATE
price	Decimal, nullable — used for SELL/RENT
rent_duration	Varchar, nullable — e.g. "per day", "per week"
condition_status	Enum: NEW, GOOD, USED
image_url	Varchar, optional
pickup_location	Varchar — e.g. Library, Hostel Gate
date_posted	Date
status	Enum: AVAILABLE, RESERVED, COMPLETED

Requests Table — one row per declared unmet need

Field	Type / Notes
request_id	Primary Key, Auto Increment
user_id	Foreign Key → users.user_id (the requester)
item_name	Varchar
category	Enum: BOOKS, ELECTRONICS, LAB_EQUIPMENT, FURNITURE, SPORTS, OTHER
required_date	Date
status	Enum: ACTIVE, FULFILLED, EXPIRED, CANCELLED
created_at	Timestamp

Transactions Table — one row per attempt to buy, rent, borrow, or claim a donation

Field	Type / Notes
transaction_id	Primary Key, Auto Increment
listing_id	Foreign Key → listings.listing_id
requester_id	Foreign Key → users.user_id (the buyer/borrower)
transaction_type	Enum: BUY, RENT, BORROW, DONATE
status	Enum: PENDING, CONFIRMED, COMPLETED, CANCELLED

created_at	Timestamp

7. Entity Relationship (ER) Diagram

The relationships below mirror the structure defined in the schema above: a user can post many listings and requests, and a listing can receive many transaction attempts.

Figure 3 — Entity relationship diagram for users, listings, requests, and transactions.

Relationship Summary

- **users** → **listings** (one-to-many): a user can post many listings.
- **users** → **requests** (one-to-many): a user can post many requests.
- **users** → **transactions** (one-to-many): a user can be the requester in many transactions.
- **listings** → **transactions** (one-to-many): a listing can receive multiple transaction attempts.